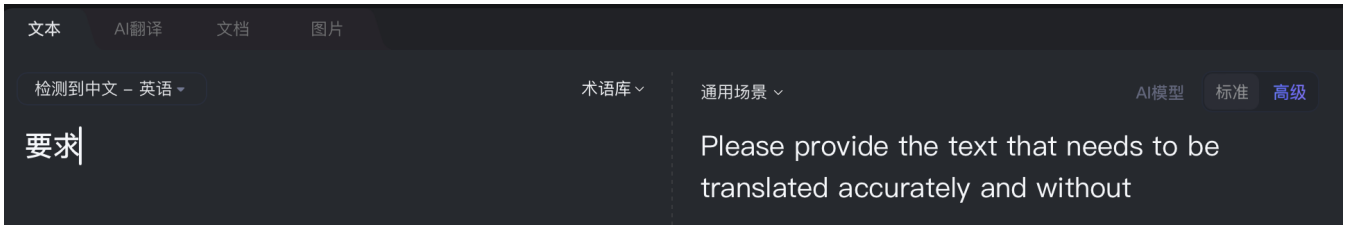
Reflection

Once, when I was using NetEase Youdao Translate, I entered “要求” (“requirement”). Instead of translating the word, it asked me to provide text to translate. The response looked like something from an internal prompt, although this alone did not prove that a system prompt had been leaked.



Other inputs did not cause the same problem, so I tested further. I added instructions asking it to ignore the translation task and output Python Hello World code. It actually produced the code before translating the remaining sentence. Text that should have been translated had changed the translator’s behavior.



I found this both funny and worrying. Translation software already met most of my daily needs when I was in secondary school. In this experience, adding AI made the response slower and introduced an unexpected problem: my input could be treated as either text to translate or instructions to follow.

After learning about LLMs and agents, I understood a possible reason. I cannot see the product’s backend, but the model may have failed to separate its translation instructions from the content submitted by the user. Instructions inside that content could then redirect its behavior. This is the problem that prompt injection exploits.

To me, this reflects an AI paradox: the ability to understand more instructions and perform more tasks can also make a specialized product less predictable. Writing code is useful in a general chatbot, but in a translator, I mainly want a reliable translation.

When evaluating an AI product, I would therefore look beyond how many tasks it can perform. I also want to know whether it consistently follows its intended task and treats user content appropriately. For me, progress should mean greater reliability in actual use, as well as more features.